WIKIPEDIA
The Free Encyclopedia

Parareal

Parareal is a parallel algorithm from numerical analysis and used for the solution of initial value problems.[1] It was introduced in 2001 by Lions, Maday and Turinici. Since then, it has become one of the most widely studied parallel-in-time integration methods.

Parallel-in-time integration methods

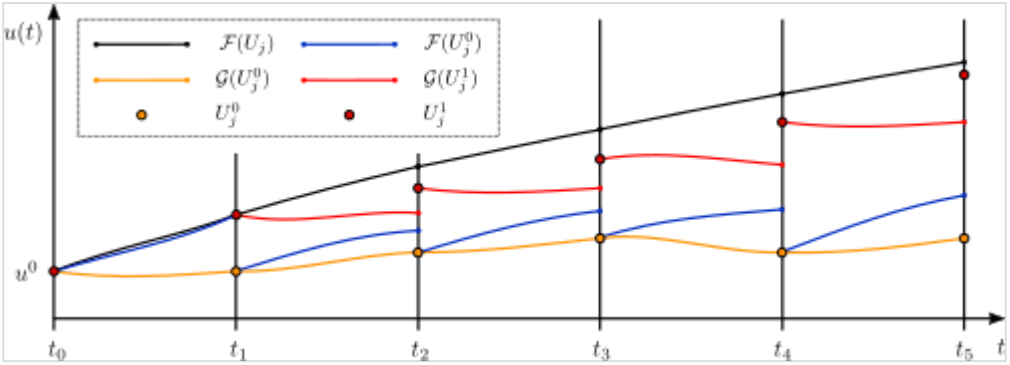


Illustration of the first iteration in Parareal (adapted from the original version[2]).

In contrast to e.g. Runge-Kutta or multi-step methods, some of the computations in Parareal can be performed in parallel and Parareal is therefore one example of a parallel-in-time integration method. While historically most efforts to parallelize the numerical solution of partial differential equations focused on the spatial discretization, in view of the challenges from exascale computing, parallel methods for temporal discretization have been identified as a possible way to increase concurrency in numerical software.[3] Because Parareal computes the numerical solution for multiple time steps in parallel, it is categorized as a *parallel across the steps* method.[4] This is in contrast to approaches using *parallelism across the method* like parallel Runge-Kutta or extrapolation methods, where independent stages can be computed in parallel or *parallel across the system* methods like waveform relaxation.[5][6]

History

Parareal can be derived as both a multigrid method in time method or as multiple shooting along the time axis.[7] Both ideas, multigrid in time as well as adopting multiple shooting for time integration, go back to the 1980s and 1990s.[8][9] Parareal is a widely studied method and has been used and modified for a range of different applications.[10] Ideas to parallelize the solution of initial value problems go back even further: the first paper proposing a parallel-in-time integration method appeared in 1964.[11]

Algorithm

The Problem

The goal is to solve an initial value problem of the form

$$\frac{\mathrm{d}u}{\mathrm{d}t} = f(t,u) \quad \text{over} \quad t \in [t_0, T] \quad \text{with} \quad u(t_0) = u^0.$$

The right hand side f is assumed to be a smooth (possibly nonlinear) function. It can also correspond to the spatial discretization of a partial differential equation in a method of lines approach. We wish to solve this problem on a temporal mesh of $N+1$ equally spaced points $(t_0, t_1, \ldots, t_N)$, where $t_{j+1} = t_j + \Delta T$ and $\Delta T = (T - t_0)/N$. Carrying out this discretisation we obtain a partitioned time interval consisting of time slices $[t_j, t_{j+1}]$ for $j = 0, \ldots, N-1$.

The objective is to calculate numerical approximations U_j to the exact solution $u(t_j)$ using a serial time-stepping method (e.g. Runge-Kutta) that has high numerical accuracy (and therefore high computational cost). We refer to this method as the fine solver $\mathcal{F}$, which propagates an initial value U_j at time t_j to a terminal value U_{j+1} at time t_{j+1}. The goal is to calculate the solution (with high numerical accuracy) using $\mathcal{F}$ such that we obtain

$$U_{j+1} = \mathcal{F}(t_j, t_{j+1}, U_j), \quad \text{where} \quad U_0 = u^0.$$

The problem with this (and the reason for attempting to solve in parallel in the first place) solution is that it is computationally infeasible to calculate in real-time.

How it works

Instead of using a single processor to solve the initial value problem (as is done with classical time-stepping methods), Parareal makes use of N processors. The aim to is to use N processors to solve N smaller initial value problems (one on each time slice) in parallel. For example, in a MPI based code, N would be the number of processes, while in an OpenMP based code, N would be equal to the number of threads.

Parareal makes use of a second time-stepping method to solve this initial value problem in parallel, referred to as the coarse solver $\mathcal{G}$. The coarse solver works the same way as the fine solver, propagating an initial value over a time interval of length ΔT, however it does so at much lower numerical accuracy than $\mathcal{F}$ (and therefore at much lower computational cost). Having a coarse solver that is much less computationally expensive than the fine solver is the key to achieving parallel speed-up with Parareal.

Henceforth, we will denote the Parareal solution at time t_j and iteration k by U_j^k.

Zeroth Iteration

Firstly, run the coarse solver serially over the entire time interval $[t_0, T]$ to calculate an approximate initial guess to the solution:

$$U_{j+1}^0 = \mathcal{G}(t_j, t_{j+1}, U_j^0), \quad j = 0, \ldots, N-1.$$

Subsequent Iterations

Next, run the fine solver on each of the time slices, in parallel, from the most up-to-date solution values:

$$U_{j+1}^1 = \mathcal{F}(t_j, t_{j+1}, U_j^0), \quad j = 0, \ldots, N-1.$$

Now update the parareal solution values sequentially using the predictor-corrector:

$$U_{j+1}^k = \mathcal{G}(t_j, t_{j+1}, U_j^k) + \mathcal{F}(t_j, t_{j+1}, U_j^{k-1}) - \mathcal{G}(t_j, t_{j+1}, U_j^{k-1}), \quad j = 0, \ldots, N-1, k = 2, 3, \ldots$$

At this stage, one can use a stopping criterion to determine whether the solution values are no longer changing each iteration. For example, one may test this by checking if

$$|U_j^k - U_j^{k-1}| < \varepsilon \quad \forall\, j \leq N,$$

and some tolerance $\varepsilon > 0$. If this critertion is not satisfied, subsequent iterations can then be run by applying the fine solver in parallel and then the predictor-corrector. Once the criterion is satisfied, however, the algorithm is said to have converged in $k \leq N$ iterations. Note that other stopping criterion do exist and have been successfully tested in Parareal.

Remarks

Parareal should reproduce the solution that is obtained by the serial application of the fine solver and will converge in a maximum of N iterations.[7] For Parareal to provide speedup, however, it has to converge in a number of iterations significantly smaller than the number of time slices, i.e. $k \ll N$.

In the Parareal iteration, the computationally expensive evaluation of $\mathcal{F}(t_j, t_{j+1}, U_j^{k-1})$ can be performed in parallel on N processing units. By contrast, the dependency of U_{j+1}^k on $\mathcal{G}(t_j, t_{j+1}, U_j^k)$ means that the coarse correction has to be computed in serial order.

Typically, some form of Runge-Kutta method is chosen for both coarse and fine integrator, where $\mathcal{G}$ might be of lower order and use a larger time step than $\mathcal{F}$. If the initial value problem stems from the discretization of a PDE, $\mathcal{G}$ can also use a coarser spatial discretization, but this can negatively impact convergence unless high order interpolation is used.[12]

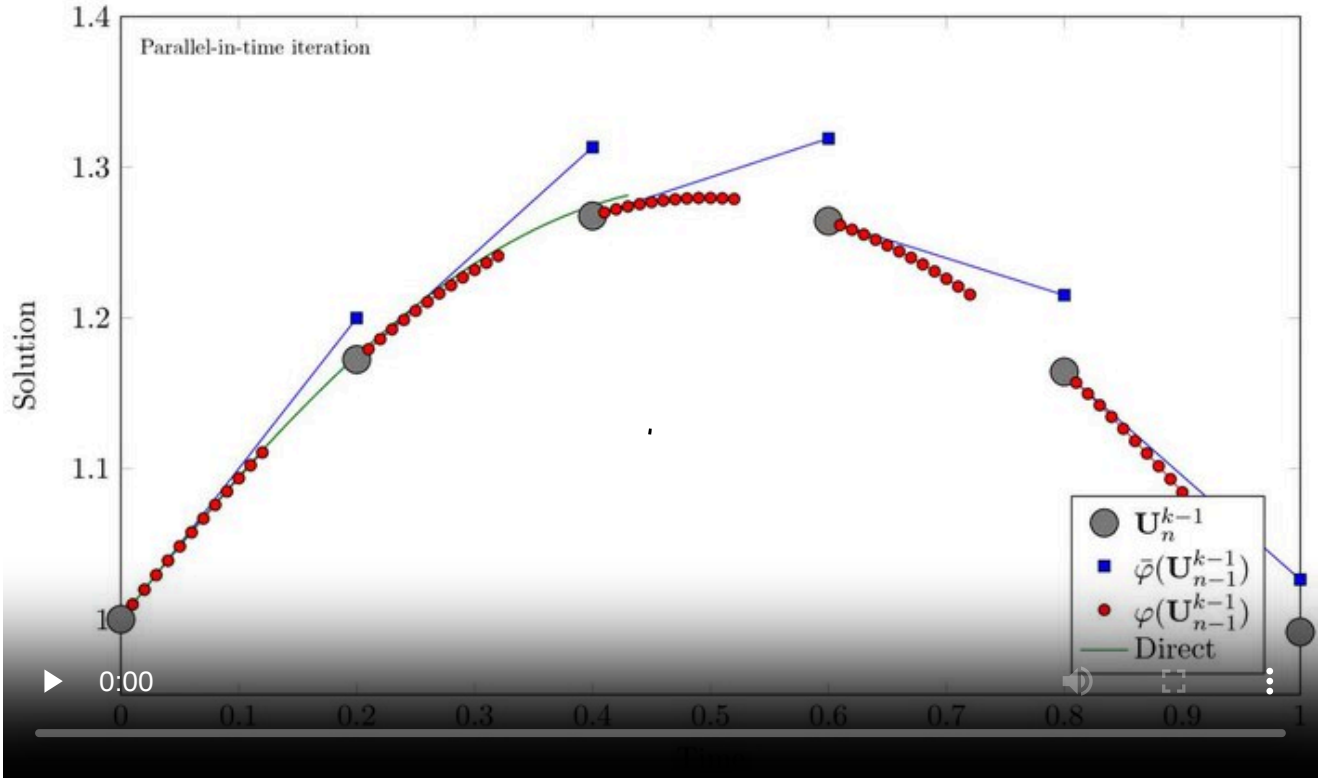
Speedup

Under some assumptions, a simple theoretical model for the speedup of Parareal can be derived.[13] Although in applications these assumptions can be too restrictive, the model still is useful to illustrate the trade offs that are involved in obtaining speedup with Parareal.

First, assume that every time slice $[t_j, t_{j+1}]$ consists of exactly N_f steps of the fine integrator and of N_c steps of the coarse integrator. This includes in particular the assumption that all time slices are of identical length and that both coarse and fine integrator use a constant step size over the full simulation. Second, denote by τ_f and τ_c the computing time required for a single step of the fine and coarse methods, respectively, and assume that both are constant. This is typically not exactly true when an implicit method is used, because then runtimes vary depending on the number of iterations required by the iterative solver.

Under these two assumptions, the runtime for the fine method integrating over P time slices can be modelled as

$$c_{\text{fine}} = PN_f\tau_f.$$



Visualization of the Parareal algorithm. The coarse propagator here is labelled $\bar{\varphi}$ whereas the fine propagator is labelled φ.

The runtime of Parareal using P processing units and performing k iterations is

$$c_{\text{parareal}} = (k+1)PN_c\tau_c + kN_f\tau_f.$$

Speedup of Parareal then is

$$S_p = \frac{c_{\text{fine}}}{c_{\text{parareal}}} = \frac{1}{(k+1)\frac{N_c}{N_f}\frac{\tau_c}{\tau_f} + \frac{k}{P}} \leq \min\left\{\frac{N_f\tau_f}{N_c\tau_c}, \frac{P}{k}\right\}.$$

These two bounds illustrate the trade off that has to be made in choosing the coarse method: on the one hand, it has to be cheap and/or use a much larger time step to make the first bound as large as possible, on the other hand the number of iterations k has to be kept low to keep the second bound large. In particular, Parareal's parallel efficiency is bounded by

$$E_p = \frac{S_p}{P} \leq \frac{1}{k},$$

that is by the inverse of the number of required iterations.

Instability for imaginary eigenvalues

The vanilla version of Parareal has issues for problems with imaginary eigenvalues.[7] It typically only converges toward the very last iterations, that is as k approaches N, and the speedup S_p is always going to be smaller than one. So either the number of iterations is small and Parareal is unstable or, if k is large enough to make Parareal stable, no speedup is possible. This also means that Parareal is typically unstable

for hyperbolic equations.[14] Even though the formal analysis by Gander and Vandewalle covers only linear problems with constant coefficients, the problem also arises when Parareal is applied to the nonlinear Navier–Stokes equations when the viscosity coefficient becomes too small and the Reynolds number too large.[15] Different approaches exist to stabilise Parareal,[16][17][18] one being Krylov-subspace enhanced Parareal.

Variants

There are multiple algorithms that are directly based or at least inspired by the original Parareal algorithm.

Krylov-subspace enhanced Parareal

Early on it was recognised that for linear problems information generated by the fine method $\mathcal{F}_{\delta t}$ can be used to improve the accuracy of the coarse method $\mathcal{G}_{\Delta t}$.[17] Originally, the idea was formulated for the parallel implicit time-integrator PITA,[19] a method closely related to Parareal but with small differences in how the correction is done. In every iteration k the result $\mathcal{F}_{\delta t}(U_j^k)$ is computed for values $u_j^k \in \mathbb{R}^d$ for $j = 0, \dots, N-1$. Based on this information, the subspace

$$S_k := \left\{ U_j^{k'} : 0 \leq k' \leq k, j = 0, \dots, N-1 \right\}$$

is defined and updated after every Parareal iteration.[20] Denote as P_k the orthogonal projection from $\mathbb{R}^d$ to S_k. Then, replace the coarse method with the improved integrator $\mathcal{K}_{\Delta t}(u) = \mathcal{F}_{\delta t}(P_k u) + \mathcal{G}_{\Delta t}((I - P_k)u)$.

As the number of iterations increases, the space S_k will grow and the modified propagator $\mathcal{K}_{\Delta t}$ will become more accurate. This will lead to faster convergence. This version of Parareal can also stably integrate linear hyperbolic partial differential equations.[21] An extension to nonlinear problems based on the reduced basis method exists as well.[18]

Hybrid Parareal spectral deferred corrections

A method with improved parallel efficiency based on a combination of Parareal with spectral deferred corrections (SDC) [22] has been proposed by M. Minion.[23] It limits the choice for coarse and fine integrator to SDC, sacrificing flexibility for improved parallel efficiency. Instead of the limit of $1/k$, the bound on parallel efficiency in the hybrid method becomes

$$E_p \leq \frac{k_s}{k_p}$$

with k_s being the number of iterations of the serial SDC base method and k_p the typically greater number of iterations of the parallel hybrid method. The Parareal-SDC hybrid has been further improved by addition of a *full approximation scheme* as used in nonlinear multigrid. This led to the development of the *parallel full approximation scheme in space and time* (PFASST).[24] Performance of PFASST has been

studied for PEPC, a Barnes-Hut tree code based particle solver developed at Juelich Supercomputing Centre. Simulations using all 262,144 cores on the IBM BlueGene/P system JUGENE showed that PFASST could produce additional speedup beyond saturation of the spatial tree parallelisation.[25]

Multigrid reduction in time (MGRIT)

The multigrid reduction in time method (MGRIT) generalises the interpretation of Parareal as a multigrid-in-time algorithms to multiple levels using different smoothers.[26] It is a more general approach but for a specific choice of parameters it is equivalent to Parareal. The XBraid (http://computation.llnl.gov/projects/parallel-time-integration-multigrid) Archived (https://web.archive.org/web/20160423223326/http://computation.llnl.gov/projects/parallel-time-integration-multigrid) 2016-04-23 at the Wayback Machine library implementing MGRIT is being developed by Lawrence Livermore National Laboratory.

ParaExp

ParaExp uses exponential integrators within Parareal.[27] While limited to linear problems, it can produce almost optimal parallel speedup.

References

1. Lions, Jacques-Louis; Maday, Yvon; Turinici, Gabriel (2015). [Lions maday (https://hal.archives-ouvertes.fr/hal-00798372/file/CRAS_01_lions_maday_turinici.pdf) "A "parareal" in time discretization of PDE's"]. *Comptes Rendus de l'Académie des Sciences, Série I*. **332** (7): 661–668. Bibcode:2001CRASM.332..661L (https://ui.adsabs.harvard.edu/abs/2001CRASM.332..661L). doi:10.1016/S0764-4442(00)01793-6 (https://doi.org/10.1016%2FS0764-4442%2800%2901793-6). {{cite journal}}: Check |url= value (help)
2. Pentland, Kamran; Tamborrino, Massimiliano; Samaddar, Debasmita; Appel, Lynton (2022). "Stochastic parareal: an application of probabilistic methods to time-parallelization" (https://wrap.warwick.ac.uk/167539/13/WRAP-stochastic-parareal-application-probabilistic-methods-time-parallelization-2023.pdf) (PDF). *SIAM Journal on Scientific Computing*. **45** (3): S82 – S102. doi:10.1137/21M1414231 (https://doi.org/10.1137%2F21M1414231). S2CID 235485544 (https://api.semanticscholar.org/CorpusID:235485544).
3. Jack Dongarra; Jeffrey Hittinger; John Bell; Luis Chacon; Robert Falgout; Michael Heroux; Paul Hovland; Esmond Ng; Clayton Webster; Stefan Wild (March 2014). Applied Mathematics Research for Exascale Computing (http://science.energy.gov/~/media/ascr/pdf/research/am/docs/EMWGreport.pdf) (PDF) (Report). US Department of Energy. Retrieved August 29, 2015.
4. Burrage, Kevin (1997). "Parallel methods for ODEs". *Advances in Computational Mathematics*. **7** (1–2): 1–31. doi:10.1023/A:1018997130884 (https://doi.org/10.1023%2FA%3A1018997130884). S2CID 15778447 (https://api.semanticscholar.org/CorpusID:15778447).
5. Iserles, A.; NøRSETT, S. P. (1990-10-01). "On the Theory of Parallel Runge—Kutta Methods". *IMA Journal of Numerical Analysis*. **10** (4): 463–488. doi:10.1093/imanum/10.4.463 (https://doi.org/10.1093%2Fimanum%2F10.4.463). ISSN 0272-4979 (https://search.worldcat.org/issn/0272-4979).

6. Ketcheson, David; Waheed, Umair bin (2014-06-13). "A comparison of high-order explicit Runge–Kutta, extrapolation, and deferred correction methods in serial and parallel". *Communications in Applied Mathematics and Computational Science*. **9** (2): 175–200. arXiv:1305.6165 (https://arxiv.org/abs/1305.6165). doi:10.2140/camcos.2014.9.175 (https://doi.org/10.2140%2Fcamcos.2014.9.175). ISSN 2157-5452 (https://search.worldcat.org/issn/2157-5452). S2CID 15242644 (https://api.semanticscholar.org/CorpusID:15242644).
7. Gander, Martin J.; Vandewalle, Stefan (2007). "Analysis of the Parareal Time-Parallel Time-Integration Method". *SIAM Journal on Scientific Computing*. **29** (2): 556–578. CiteSeerX 10.1.1.154.6042 (https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.154.6042). doi:10.1137/05064607X (https://doi.org/10.1137%2F05064607X).
8. Hackbusch, Wolfgang (1985). *Parabolic multi-grid methods* (http://dl.acm.org/citation.cfm?id=4673.4714). pp. 189–197. ISBN 9780444875976. Retrieved August 29, 2015. {{cite book}}: |journal= ignored (help)
9. Kiehl, Martin (1994). "Parallel multiple shooting for the solution of initial value problems". *Parallel Computing*. **20** (3): 275–295. doi:10.1016/S0167-8191(06)80013-X (https://doi.org/10.1016%2FS0167-8191%2806%2980013-X).
10. Gander, Martin J. (2015). *50 years of Time Parallel Time Integration*. Contributions in Mathematical and Computational Sciences. Vol. 9 (1 ed.). Springer International Publishing. doi:10.1007/978-3-319-23321-5 (https://doi.org/10.1007%2F978-3-319-23321-5). ISBN 978-3-319-23321-5.
11. Nievergelt, Jürg (1964). "Parallel methods for integrating ordinary differential equations" (https://doi.org/10.1145%2F355588.365137). *Communications of the ACM*. **7** (12): 731–733. doi:10.1145/355588.365137 (https://doi.org/10.1145%2F355588.365137). S2CID 6361754 (https://api.semanticscholar.org/CorpusID:6361754).
12. Ruprecht, Daniel (2014-12-01). "Convergence of Parareal with spatial coarsening" (http://eprints.whiterose.ac.uk/90536/1/paper.pdf) (PDF). *Proceedings in Applied Mathematics and Mechanics*. **14** (1): 1031–1034. doi:10.1002/pamm.201410490 (https://doi.org/10.1002%2Fpamm.201410490). ISSN 1617-7061 (https://search.worldcat.org/issn/1617-7061). S2CID 26356904 (https://api.semanticscholar.org/CorpusID:26356904).
13. Minion, Michael L. (2010). "A Hybrid Parareal Spectral Deferred Corrections Method" (https://doi.org/10.2140%2Fcamcos.2010.5.265). *Communications in Applied Mathematics and Computational Science*. **5** (2): 265–301. doi:10.2140/camcos.2010.5.265 (https://doi.org/10.2140%2Fcamcos.2010.5.265).
14. Staff, Gunnar Andreas; Rønquist, Einar M. (2005-01-01). Barth, Timothy J.; Griebel, Michael; Keyes, David E.; Nieminen, Risto M.; Roose, Dirk; Schlick, Tamar; Kornhuber, Ralf; Hoppe, Ronald; Périaux, Jacques (eds.). *Stability of the Parareal Algorithm*. Lecture Notes in Computational Science and Engineering. Springer Berlin Heidelberg. pp. 449–456. doi:10.1007/3-540-26825-1_46 (https://doi.org/10.1007%2F3-540-26825-1_46). ISBN 9783540225232.
15. Steiner, Johannes; Ruprecht, Daniel; Speck, Robert; Krause, Rolf (2015-01-01). "Convergence of Parareal for the Navier-Stokes Equations Depending on the Reynolds Number". In Abdulle, Assyr; Deparis, Simone; Kressner, Daniel; Nobile, Fabio; Picasso, Marco (eds.). *Numerical Mathematics and Advanced Applications - ENUMATH 2013*. Lecture Notes in Computational Science and Engineering. Vol. 103. Springer International Publishing. pp. 195–202. CiteSeerX 10.1.1.764.6242 (https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.764.6242). doi:10.1007/978-3-319-10705-9_19 (https://doi.org/10.1007%2F978-3-319-10705-9_19). ISBN 9783319107042.

16. Dai, X.; Maday, Y. (2013-01-01). "Stable Parareal in Time Method for First- and Second-Order Hyperbolic Systems". *SIAM Journal on Scientific Computing*. **35** (1): A52 – A78. arXiv:1201.1064 (https://arxiv.org/abs/1201.1064). Bibcode:2013SJSC...35A..52D (https://ui.adsabs.harvard.edu/abs/2013SJSC...35A..52D). doi:10.1137/110861002 (https://doi.org/10.1137%2F110861002). ISSN 1064-8275 (https://search.worldcat.org/issn/1064-8275). S2CID 32336370 (https://api.semanticscholar.org/CorpusID:32336370).
17. Farhat, Charbel; Cortial, Julien; Dastillung, Climène; Bavestrello, Henri (2006-07-30). "Time-parallel implicit integrators for the near-real-time prediction of linear structural dynamic responses". *International Journal for Numerical Methods in Engineering*. **67** (5): 697–724. Bibcode:2006IJNME..67..697F (https://ui.adsabs.harvard.edu/abs/2006IJNME..67..697F). doi:10.1002/nme.1653 (https://doi.org/10.1002%2Fnme.1653). ISSN 1097-0207 (https://search.worldcat.org/issn/1097-0207). S2CID 121254646 (https://api.semanticscholar.org/CorpusID:121254646).
18. Chen, Feng; Hesthaven, Jan S.; Zhu, Xueyu (2014-01-01). "On the Use of Reduced Basis Methods to Accelerate and Stabilize the Parareal Method". In Quarteroni, Alfio; Rozza, Gianluigi (eds.). *Reduced Order Methods for Modeling and Computational Reduction* (http://infoscience.epfl.ch/record/190666). MS&A - Modeling, Simulation and Applications. Springer International Publishing. pp. 187–214. doi:10.1007/978-3-319-02090-7_7 (https://doi.org/10.1007%2F978-3-319-02090-7_7). ISBN 9783319020891.
19. Farhat, Charbel; Chandesris, Marion (2003-11-07). "Time-decomposed parallel time-integrators: theory and feasibility studies for fluid, structure, and fluid–structure applications". *International Journal for Numerical Methods in Engineering*. **58** (9): 1397–1434. Bibcode:2003IJNME..58.1397F (https://ui.adsabs.harvard.edu/abs/2003IJNME..58.1397F). doi:10.1002/nme.860 (https://doi.org/10.1002%2Fnme.860). ISSN 1097-0207 (https://search.worldcat.org/issn/1097-0207). S2CID 61667246 (https://api.semanticscholar.org/CorpusID:61667246).
20. Gander, M.; Petcu, M. (2008). "Analysis of a Krylov subspace enhanced parareal algorithm for linear problems" (https://doi.org/10.1051%2Fproc%3A082508). *ESAIM: Proceedings*. **25**: 114–129. doi:10.1051/proc:082508 (https://doi.org/10.1051%2Fproc%3A082508).
21. Ruprecht, D.; Krause, R. (2012-04-30). "Explicit parallel-in-time integration of a linear acoustic-advection system". *Computers & Fluids*. **59**: 72–83. arXiv:1510.02237 (https://arxiv.org/abs/1510.02237). doi:10.1016/j.compfluid.2012.02.015 (https://doi.org/10.1016%2Fj.compfluid.2012.02.015). S2CID 15703896 (https://api.semanticscholar.org/CorpusID:15703896).
22. Dutt, Alok; Greengard, Leslie; Rokhlin, Vladimir (2000-06-01). "Spectral Deferred Correction Methods for Ordinary Differential Equations". *BIT Numerical Mathematics*. **40** (2): 241–266. doi:10.1023/A:1022338906936 (https://doi.org/10.1023%2FA%3A1022338906936). ISSN 0006-3835 (https://search.worldcat.org/issn/0006-3835). S2CID 43449672 (https://api.semanticscholar.org/CorpusID:43449672).
23. Minion, Michael (2011-01-05). "A hybrid parareal spectral deferred corrections method" (https://doi.org/10.2140%2Fcamcos.2010.5.265). *Communications in Applied Mathematics and Computational Science*. **5** (2): 265–301. doi:10.2140/camcos.2010.5.265 (https://doi.org/10.2140%2Fcamcos.2010.5.265). ISSN 2157-5452 (https://search.worldcat.org/issn/2157-5452).
24. Emmett, Matthew; Minion, Michael (2012-03-28). "Toward an efficient parallel in time method for partial differential equations" (https://doi.org/10.2140%2Fcamcos.2012.7.105). *Communications in Applied Mathematics and Computational Science*. **7** (1): 105–132. doi:10.2140/camcos.2012.7.105 (https://doi.org/10.2140%2Fcamcos.2012.7.105). ISSN 2157-5452 (https://search.worldcat.org/issn/2157-5452).

25. Speck, R.; Ruprecht, D.; Krause, R.; Emmett, M.; Minion, M.; Winkel, M.; Gibbon, P. (2012-11-01). "A massively space-time parallel N-body solver". *2012 International Conference for High Performance Computing, Networking, Storage and Analysis*. pp. 1–11. doi:10.1109/SC.2012.6 (https://doi.org/10.1109%2FSC.2012.6). ISBN 978-1-4673-0805-2. S2CID 1620219 (https://api.semanticscholar.org/CorpusID:1620219).
26. Falgout, R.; Friedhoff, S.; Kolev, T.; MacLachlan, S.; Schroder, J. (2014-01-01). "Parallel Time Integration with Multigrid". *SIAM Journal on Scientific Computing*. **36** (6): C635 – C661. Bibcode:2014SJSC...36C.635F (https://ui.adsabs.harvard.edu/abs/2014SJSC...36C.635F). CiteSeerX 10.1.1.701.2603 (https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.701.2603). doi:10.1137/130944230 (https://doi.org/10.1137%2F130944230). ISSN 1064-8275 (https://search.worldcat.org/issn/1064-8275).
27. Gander, M.; Güttel, S. (2013-01-01). "PARAEXP: A Parallel Integrator for Linear Initial-Value Problems". *SIAM Journal on Scientific Computing*. **35** (2): C123 – C142. Bibcode:2013SJSC...35C.123G (https://ui.adsabs.harvard.edu/abs/2013SJSC...35C.123G). CiteSeerX 10.1.1.800.5938 (https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.800.5938). doi:10.1137/110856137 (https://doi.org/10.1137%2F110856137). ISSN 1064-8275 (https://search.worldcat.org/issn/1064-8275).

External links

- parallel-in-time.org (https://www.parallel-in-time.org/)

Retrieved from "https://en.wikipedia.org/w/index.php?title=Parareal&oldid=1307736457"